

Cours 3

Chemins, fichiers et ligne de commande, en Python

1re année géomatique
29 septembre 2026

Objectifs de la séance

Manipuler des chemins et lire des fichiers en Python.

Construire un programme en ligne de commande en Python.

Revoir les notions des cours 1 et 2 côté code Python.

Vu aux cours 1 et 2	Aujourd'hui, en Python
chemin relatif, chemin absolu	<code>Path</code> , <code>/</code> , <code>resolve()</code> , <code>Path(__file__)</code>
fichier texte, encodage (cours 1)	<code>open(file_path, encoding="utf-8")</code> , <code>fichier.write(...)</code>
texte et binaire (cours 1)	<code>open(file_path, "wb", ...)</code> , une image écrite en format texte et binaire
lancer un programme au terminal (cours 2)	<code>subprocess.run(...)</code> , avec <code>pandoc md -> html</code>
Ecrire une ligne de commande	<code>argparse</code> , dans un programme qu'on écrit

Contenu de la séance

La séance contient trois notebooks ; les deux premiers s'exécutent pendant la présentation du cours. Chaque TD a son dossier dans l'archive `cours3/`, avec ses propres données.

Partie	Fichier	Contenu	Durée
Préparation du poste de travail		récupérer l'archive, lancer JupyterLab	10 min
Environnement (selon les groupes)	TD 0a	<code>cours3</code> , avec pandoc et Pillow	+15 min
Chemins	<code>recette.ipynb</code>	chemins ; appeler une commande depuis Python	20 min
Texte et binaire	<code>fichiers.ipynb</code> <code>images.ipynb</code>	lire et écrire un fichier ; texte et binaire ; UTF-8	30 min
Ligne de commande	<code>recette.py</code>	<code>main</code> , <code>argparse</code> ; un commit par étape	45 min

Lancement de la séance : récupération données

L'archive `info01-cours3.zip` est dans le dossier partagé `formationTemp`. Il faut la copier sur le Bureau (ou autre dossier de votre préférence) puis la décompresser.

	Ce qu'il faut faire	Ce que vous devez obtenir
1	ouvrir le dossier partagé <code>formationTemp</code> , copier <code>info01-cours3.zip</code> sur le Bureau	l'archive sur le Bureau, 14 Mo
2	clic droit sur l'archive → Extraire tout	un dossier <code>cours3/</code> , quatre sous-dossiers <code>1a_recette/</code> , <code>2a_fichiers/</code> , <code>2b_images/</code> , <code>3a_cli/</code>
3	ouvrir <code>cours3/1a_recette/</code> : <code>depart/</code> , <code>travail/</code> vide, la feuille du TD	<code>depart/notebook/</code> contient <code>recette.ipynb</code>

Attention. Ne pas travailler dans le dossier partagé, ne pas chercher à ouvrir un fichier de l'archive avec vscode ou autre.

Lancement de la séance : test notebook

La séance demande d'exécuter des notebooks et d'écrire du code Python. Avant de commencer, chacun vérifie que ses outils se lancent.

Par défaut on utilisera vscode ou jupyter lab. Si ça ne fonctionne pas demandez, selon les cas ça sera dépannage ou changement de poste.

Outil	Comment le lancer	Ce qui doit apparaître
JupyterLab	Anaconda Navigator → fiche JupyterLab → Launch ; si rien ne vient après trente secondes : Anaconda Prompt, puis <code>jupyter lab</code>	un onglet du navigateur, adresse <code>localhost</code>
Un éditeur	VS Code, configuré au TD 2a du cours 1 ; sinon Spyder, depuis Navigator	l'éditeur, avec un terminal qui répond à <code>python --version</code>
Un terminal	menu Démarrer → Anaconda Prompt	<code>(base)</code> en tête de ligne

En cas de problème : les pages « Avant les séances » du support, ou la main levée.

Un environnement pour la séance

Créer l'environnement cours3, y installer les paquets que demandent les notebooks, et y lancer JupyterLab

`cours3/0a_environnement/` `≈ 20'`

Un environnement conda

Un environnement conda est un dossier qui contient un Python et des paquets. Chaque projet a le sien : installer un paquet pour un projet ne change pas les autres.

Commande	Ce qu'elle fait
<code>conda create -n cours3 -c conda-forge python=3.12</code>	crée l'environnement <code>cours3</code> , avec Python
<code>conda activate cours3</code>	le terminal emploie le Python et les programmes de <code>cours3</code> ; l'invite commence par <code>(cours3)</code>
<code>conda install -c conda-forge pandoc</code>	installe un paquet dans l'environnement actif
<code>conda list</code>	liste les paquets de l'environnement actif
<code>conda env list</code>	liste les environnements du poste ; * marque l'environnement actif

`-n` donne le nom de l'environnement. `-c conda-forge` donne le dépôt d'où viennent les paquets.

Créer l'environnement

Dans l'Anaconda Prompt. L'environnement contient Python et JupyterLab, et rien d'autre.

	Ce qu'il faut faire	Ce que vous constatez
1	<code>conda create -n cours3 -c conda-forge python=3.12 jupyterlab, puis y</code>	
2	<code>conda activate cours3</code>	
3	<code>pandoc --version</code>	
4	<code>python -c "import PIL"</code>	

`python -c` exécute le code écrit entre guillemets. Pillow s'importe sous le nom `PIL`.

Installer les paquets demandés

`recette.ipynb` appelle `pandoc` ; `images.ipynb` importe `Pillow`. On les installe dans `cours3`, puis on y lance JupyterLab.

	Ce qu'il faut faire	Ce que vous constatez
5	<code>conda install -c conda-forge pandoc pillow</code> , puis <code>y</code>	_____
6	<code>pandoc --version</code> , puis <code>python -c "import PIL"</code>	_____
7	<code>cd</code> vers le dossier <code>cours3/</code> extrait sur le Bureau, puis <code>jupyter lab</code>	_____

Le terminal qui a lancé `jupyter lab` reste ouvert jusqu'à la fin de la séance : c'est le serveur des notebooks. Les notebooks s'exécutent dans `cours3`.

Chemins

Améliorer le code de génération de recette : ses chemins avec pathlib, sa conversion avec pandoc

TD 1a

Notebook à suivre pendant l'exposé

`recette.ipynb`

`cours3/1a_recette/` $\approx 5'$

Ouvrir le notebook

Les notebooks sont livrés dans `depart/notebook/`. On travaille sur une copie, dans `travail/` : `depart/` ne se modifie pas.

	Ce qu'il faut faire
1	dans l'explorateur, copier <code>depart/notebook/recette.ipynb</code> dans <code>travail/</code>
2	dans le panneau de gauche de JupyterLab, descendre jusqu'à <code>cours3/1a_recette/travail/</code> , double-cliquer sur <code>recette.ipynb</code>
3	ouvrir aussi <code>depart/recettes/crepes/recette.md</code> par un double-clic

Les fichiers créés par le notebook sont écrits dans `travail/`, à côté de lui.

Le code brut, chemins en dur

§ 1 et 2

Le programme de création de recette, avec des chemins écrits en dur : il ne fonctionne que sur un poste. Les fonctions utiles du bloc 1 sont utilisées sans détailler leur code, vu dans le second notebook.

```
ingredients = lire_ingredients("C:/Users/alice/.../ingredients.csv")
ingredients = adapter(ingredients, 4, "SI")
with open("C:/Users/alice/.../recette.md", encoding="utf-8") as fichier:
    source = fichier.read()
complete = source.replace("## Ingrédients",
    "## Ingrédients\n\n" + tableau(ingredients))
with open("C:/Users/alice/.../crepes.md", "w", encoding="utf-8") as fichier:
    fichier.write(complete)
```

lit le CSV
4 personnes, SI
ouvre la recette
tout son texte
insère le tableau
ouvre la sortie
l'écrit

Attention. Ces chemins correspondent à ceux d'un poste spécifique. Sur un autre, ou après un déplacement du dossier, les trois lignes sont donc à adapter.

Lire le fichier CSV

`ingredients.csv` donne les quantités pour une personne, une ligne par ingrédient. `lire_ingredients` lit le fichier et renvoie une liste Python : un tuple (nom, quantité, unité) par ligne.

`recettes/crepes/ingredients.csv`

`lire_ingredients(...)` renvoie

```
ingredient,quantite,unite
Farine,60,g
Lait,125,ml
Œufs,1,
Sel,1,g
Beurre fondu,12,g
```

```
[('Farine', 60.0, 'g'),
 ('Lait', 125.0, 'ml'),
 ('Œufs', 1.0, ''),
 ('Sel', 1.0, 'g'),
 ('Beurre fondu', 12.0, 'g')]
```

La première ligne du fichier, le nom des colonnes, n'est pas dans la liste. `float` convertit chaque quantité, lue comme du texte, en nombre.

Adapter les quantités, écrire le tableau

`adapter` multiplie chaque quantité par le nombre de personnes ; en unités US, il convertit les grammes en onces et les millilitres en tasses. `tableau` écrit la liste sous forme de tableau Markdown.

`adapter(ingredients, 4, "US")` renvoie

```
[('Farine', 8.4657..., 'oz'),  
 ('Lait', 2.1133..., 'cup'),  
 ('Œufs', 4.0, ''),  
 ('Sel', 0.1410..., 'oz'),  
 ('Beurre fondu', 1.6931..., 'oz')]
```

`tableau(...)` renvoie

```
| Ingrédient | Quantité |  
|---|---|  
| Farine | 8.47 oz |  
| Lait | 2.11 cup |  
| Œufs | 4 |  
| Sel | 0.141 oz |  
| Beurre fondu | 1.69 oz |
```

En unités SI, pour quatre personnes : 240 g de farine, 500 ml de lait. `tableau` écrit chaque quantité avec trois chiffres significatifs.

Insérer le tableau dans la recette

Dans `recette.md`, le titre `## Ingrédients` n'est suivi d'aucune ligne. `replace` le remplace par le même titre suivi du tableau. Le texte obtenu est écrit dans `travail/crepes.md`.

`recette.md`

```
# Crêpes

## Ingrédients

## Préparation
```

+

`tableau(...)`

```
| Ingrédient | Quantité |
|---|---|
| Farine | 240 g |
| Lait | 500 ml |
| ...
```

→

`travail/crepes.md`

```
# Crêpes

## Ingrédients

| Ingrédient | Quantité |
|---|---|
| Farine | 240 g |
| ...

## Préparation
```

```
complete = source.replace("## Ingrédients", "## Ingrédients\n\n" + tableau(ingredients))
```

le titre, une ligne
vide, le tableau

Les chemins à déclarer

§ 3.1 et 3.2

Le code s'améliore en deux étapes :

1. utiliser des variables pour les chemins de fichier (python pur).
2. déduire l'ensemble des chemins à partir d'une seule variable : dossier racine avec `pathlib`.

```
1a_recette/          ← RACINE
├── depart/
│   ├── recettes/
│   │   ├── crepes/      ← RECETTE
│   │   │   ├── ingredients.csv ← FICHIER_INGREDIENTS
│   │   │   └── recette.md   ← FICHIER_RECETTE
│   └── travail/         ← dossier courant
│       ├── recette.ipynb  ← le notebook
│       └── crepes.md      ← FICHIER_SORTIE
```

Seule la racine est écrite en dur ; les quatre autres chemins sont construits à partir d'elle, en suivant l'arborescence.

pathlib pour déclarer des chemins

§ 3.2

```
from pathlib import Path
RACINE = Path("C:/Users/alice/.../la_recette")
RECETTE = RACINE / "depart" / "recettes" / "crepes"
FICHER_INGREDIENTS = RECETTE / "ingredients.csv"
FICHER_RECETTE = RECETTE / "recette.md"
FICHER_SORTIE = RACINE / "travail" / "crepes.md"
ingredients = lire_ingredients(FICHER_INGREDIENTS)
```

Documentation : docs.python.org/fr/3/library/pathlib.html.

importe Path, la classe des chemins

la racine, seule valeur écrite en dur

/ ajoute un dossier au chemin

le CSV

la recette

le fichier produit

le code utilise les variables

La racine, lue automatiquement

§ 3.3

Python connaît le dossier courant, celui d'où partent les chemins relatifs. Dans un notebook, c'est le dossier du fichier `.ipynb`, ici `travail/` ; la racine du TD est le dossier au-dessus.

```
Path.cwd()
sys.executable
(Path.cwd() / "recette.ipynb").exists()
Path.cwd().parent
Path("../") / "depart" / "recettes"
(Path("../") / "depart" / "recettes").resolve()
RACINE = Path.cwd().parent
```

le dossier courant : `.../la_recette/travail`
l'interpréteur Python, ailleurs
True : le notebook est bien dans le dossier courant
le dossier au-dessus, en absolu : la racine du TD
le même endroit en relatif, affiché tel qu'écrit : `../`
`depart/recettes`
`resolve()` le colle au dossier courant : le chemin absolu
plus rien n'est écrit en dur

Plusieurs recettes : lister un dossier

§ 3.4

Le code actuel ne traite qu'une recette : `crepes`.

`depart/recettes/` contient quatre dossiers de recette construits de façon identique : le programme les parcourt, et nomme chaque sortie d'après le nom du dossier.

```
la_recette/          ← RACINE
├─ depart/
│   └─ recettes/      ← RECETTES
│       ├── crepes/    ← dossier, à la première itération
│       │   ├── ingredients.csv
│       │   └─ recette.md
│       ├── ...
│       └─ salade_lentilles/ ← dossier, à la dernière itération
│           ├── ingredients.csv
│           └─ recette.md
└─ travail/           ← dossier courant
    ├── recette.ipynb  ← le notebook
    ├── crepes.md      ← fichier produit à la première itération
    ├── ...
    └─ salade_lentilles.md ← fichier produit à la dernière itération
```

Plusieurs recettes : lister un dossier

§ 3.4

```
RECETTES = RACINE / "depart" / "recettes"
for dossier in sorted(RECETTES.iterdir()):
    if dossier.is_dir():
        dossier.name
        dossier / "ingredients.csv"
        (RACINE / "travail" / dossier.name).with_suffix(".md")
        (dossier / "ingredients.csv").exists()
```

le dossier des recettes
chaque entrée du dossier, triée
seulement les dossiers
le nom seul : crepes
le CSV de cette recette
le fichier produit : crepes.md
True si le fichier est là

Les parties d'un chemin

§ 3.4

Les attributs d'un objet `Path` renvoient les parties du chemin : le nom du fichier, son extension, le dossier qui le contient. Exemples avec `chemin = RECETTES / "crepes" / "recette.md"`.

<code>chemin.name</code>	le nom du fichier : <code>recette.md</code>
<code>chemin.stem</code>	le nom sans l'extension : <code>recette</code>
<code>chemin.suffix</code>	l'extension, point compris : <code>.md</code>
<code>chemin.parent</code>	le dossier qui le contient : <code>.../recettes/crepes</code>
<code>chemin.parent.name</code>	le nom de ce dossier : <code>crepes</code>
<code>chemin.parts</code>	toutes les parties du chemin, dans un tuple
<code>chemin.relative_to(RACINE)</code>	le chemin à partir de la racine : <code>depart/recettes/crepes/recette.md</code>
<code>chemin.with_suffix(".html")</code>	même chemin, autre extension : <code>.../crepes/recette.html</code>
<code>chemin.with_name("ingredients.csv")</code>	même dossier, autre nom : <code>.../crepes/ingredients.csv</code>

pandoc dans le terminal

§ 4.1

pandoc convertit `crepes.md`, dans `travail/`, en page HTML. C'est un programme : il se lance dans le terminal.

```
(base) C:\Users\moi> cd Desktop\cours3\la_recette\travail
(base) C:\Users\moi\Desktop\cours3\la_recette\travail> pandoc crepes.md -o crepes.html
```

pandoc	le programme
<code>crepes.md</code>	l'entrée : le fichier à lire
<code>-o crepes.html</code>	- - output, la sortie ; le format suit l'extension

Depuis `cours3/`, entrée et sortie partent de ce dossier :

```
(base) C:\Users\moi\Desktop\cours3> pandoc la_recette\travail\crepes.md -o
la_recette\travail\crepes.html
```

Documentation : pandoc.org/MANUAL.html ; exemples : pandoc.org/demos.html.

pandoc depuis le notebook : les options une à une

§ 4.2

Une cellule qui commence par `!` passe la ligne au terminal. La même commande, une option de plus à chaque fois.

Option ajoutée	Ce qui change
(aucune)	un fragment : le contenu, sans <code><html></code> ni en-tête
<code>--standalone</code>	une page complète ; titre pris sur le nom du fichier
<code>--metadata title=Crêpes</code>	le titre de l'onglet
<code>--css style.css</code>	la feuille de style, copiée à côté de la page
<code>--toc</code>	une table des matières, sur les titres
<code>-o crepes.docx, -t plain</code>	Word ; texte sans balise
<code>pandoc crepes.docx -o retour.md</code>	le sens inverse

Où le terminal trouve pandoc

§ 4.3

Le terminal a lancé `pandoc` sans savoir où est le programme : il l'a cherché dans les dossiers de la variable `PATH`.

```
shutil.which("pandoc")  
!"{chemin}" --version
```

le chemin absolu trouvé

le même programme, appelé par ce chemin

```
path = os.environ["PATH"]  
os.pathsep
```

la variable : une seule chaîne, tous les dossiers

le séparateur : `;` sous Windows, `:` ailleurs

```
dossiers = path.split(os.pathsep)
```

une liste, un dossier par élément

```
Path(dossier) == Path(chemin).parent
```

vrai pour le dossier qui contient pandoc

Une commande « introuvable » est un programme dont le dossier n'est pas dans cette liste. `conda activate` modifie `PATH`.

Appeler pandoc depuis Python

§ 4.4

! n'existe que dans un notebook. Un programme Python appelle une commande par la bibliothèque standard : `os.system`, OU `subprocess.run`, la forme à retenir.

```
os.system("pandoc crepes.md -o crepes.html")
```

la ligne telle qu'on l'aurait tapée ;
renvoie le code de retour, 0 si tout
va bien

```
subprocess.run(["pandoc", "crepes.md", "-o", "crepes.html"],  
               check=True)
```

le programme, puis chaque
argument, en liste
lève une erreur Python si pandoc
échoue

```
resultat = subprocess.run(["pandoc", "--version"],  
                           capture_output=True, text=True)  
resultat.returncode  
resultat.stdout
```

récupère l'affichage, en chaîne
0 : tout s'est bien passé
ce que pandoc a affiché

Un `Path` se passe par `str(chemin)` : la commande ne prend que des chaînes. C'est la forme du TD 3a.

Texte et binaire

Comment le code ouvre, lit et écrit ses fichiers texte ; puis, sur des images au format PGM, ce qu'un fichier binaire contient et ce que le format change au poids et au temps de lecture

TD 2a

Notebook à suivre pendant l'exposé

`fichiers.ipynb`

`cours3/2a_fichiers/` `≈ 15'`

Ouvrir le notebook des fichiers

Les fonctions utiles de `recette.ipynb` ouvrent, lisent et écrivent des fichiers sans que ces lignes aient été expliquées. `fichiers.ipynb` reprend le code dans son dernier état et explique ces lignes. Il est dans le dossier `cours3/2a_fichiers/`, avec une copie des recettes.

	Ce qu'il faut faire	Ce que vous constatez
1	dans l'explorateur, copier <code>2a_fichiers/depart/notebook/fichiers.ipynb</code> dans <code>2a_fichiers/travail/</code>	_____
2	dans le panneau de gauche de JupyterLab, double-cliquer dessus	_____
3	exécuter la section 0	_____

`recette.ipynb` peut rester ouvert. Le notebook des images, dans `2b_images/`, s'ouvre au TD 2b.

Ouvrir, lire, fermer

fichiers.ipynb · § 1

`open` ouvre le fichier et renvoie un objet qui permet de le lire. La méthode `read()` de cet objet renvoie le contenu du fichier. Ici, l'objet est nommé `fichier_ouvert` et le contenu lu `texte`.

```
fichier_ouvert = open(FICHER_RECETTE, encoding="utf-8")
texte = fichier_ouvert.read()
fichier_ouvert.read()
fichier_ouvert.close()
fichier_ouvert.read()
```

```
repr(texte[:60])
```

```
lignes = fichier_ouvert.readlines()
```

`ouvre` : un objet fichier, position au début

`lit tout, jusqu'à la fin` : une seule chaîne

`' '` : la position est à la fin

`ferme` : le fichier est rendu au système

`ValueError` : fichier fermé

les caractères tels qu'ils sont : `\n` entre les

lignes

une liste : une chaîne par ligne, avec son `\n`

with : ouvrir et fermer

fichiers.ipynb · § 2

Le bloc `with` ouvre le fichier à l'entrée et le ferme à la sortie, y compris si une erreur survient dans le bloc. C'est la forme à employer.

```
with open(FICHER_RECETTE, encoding="utf-8") as fichier_ouvert:
```

```
    texte = fichier_ouvert.read()
```

```
print(fichier_ouvert.closed)
```

ouvert, disponible sous le
nom `fichier_ouvert`
tout ce qui se fait fichier
ouvert est indenté

True : à la sortie du bloc,
déjà fermé

Les modes d'ouverture

fichiers.ipynb · § 3

Le deuxième argument d'`open` indique le mode d'ouverture du fichier. Par défaut, le fichier est ouvert en lecture seule (mode `"r"`).

Mode	Ce qu'il fait	Fichier absent	Fichier présent
"r"	lire (défaut)	erreur	lu
"w"	écrire	créé	vidé, puis réécrit
"a"	ajouter à la fin	créé	conservé, complété
"x"	créer et écrire	créé	erreur
"rb", "wb"	octets, sans <code>encoding</code>		notebook des images

Attention. Sans `encoding="utf-8"`, Python prend l'encodage du système, `cp1252` sous Windows : « é » écrit en UTF-8 se lit « Ã© ».

Lire ligne par ligne

fichiers.ipynb · § 4

`read()` et `readlines()` lisent tout le fichier en une fois. On peut aussi lire le fichier ligne par ligne avec une boucle `for` : une ligne est lue à chaque itération. Pour un gros fichier, cela évite de charger tout le contenu en mémoire.

```
with open(FICHIER_INGREDIENTS, encoding="utf-8") as fichier_ouvert:
    for ligne in fichier_ouvert:
        print(repr(ligne))
```

une ligne à la fois, avec son `\n`
'nom,quantite,unite\n',
puis chaque ingrédient

```
with open(FICHIER_RECETTE, encoding="utf-8") as fichier_ouvert:
    numero = 0
    for ligne in fichier_ouvert:
        numero = numero + 1
        print(numero, ligne.strip())
        if numero == 3:
            break
```

compte les lignes lues
`strip()` enlève le `\n`

sort de la boucle : la suite n'est
pas lue

À lire après la séance : § 5 et 6

Les deux dernières sections de `fichiers.ipynb` se lisent et s'exécutent après la séance. `images.ipynb` et le TD 3a emploient les méthodes de la section 6.

Section	Contenu	Ce qui sert ensuite
§ 5 · Le CSV	découper une ligne aux virgules avec <code>split</code> , puis avec la bibliothèque <code>csv</code>	<code>lire_ingredients</code> de la recette
§ 6 · Les raccourcis de <code>pathlib</code>	<code>read_text</code> , <code>write_text</code> , <code>read_bytes</code> : ouvrir, lire ou écrire tout le fichier, fermer, en une ligne	<code>images.ipynb</code> (§ 1 et 2) ; le TD 3a

```
texte = chemin.read_text(encoding="utf-8")
octets = chemin.read_bytes()
```

le même résultat que `with open(...) puis read()`
les octets du fichier, sans décodage

TD 2b

Texte et binaire : les images PGM

Ouvrir `images.ipynb` ; le notebook se fait pendant l'exposé,
section par section

`cours3/2b_images/` $\approx 15'$

Exemple de fichier binaire : les images PGM

Le format d'image PGM existe en deux variantes : texte et binaire. Le notebook `images.ipynb` compare les deux fichiers d'une même image.

Ouvrir le notebook `images.ipynb` du dossier `cours3/2b_images/`.

`depart/` contient trois images : un motif de seize pixels, **La Grande Vague** de Hokusai et un émoji. Les fichiers créés par le notebook sont écrits dans `travail/`.

Le format PGM

Un fichier PGM est une image en niveaux de gris : un en-tête de trois lignes, puis une valeur par pixel, 0 pour le noir. Ses deux variantes ne diffèrent que par l'écriture des pixels.

	P2, texte	P5, binaire
ligne 1 : le nom du format	P2	P5
ligne 2 : largeur et hauteur	4 4	4 4
ligne 3 : la valeur du blanc, 1 à 65535	255	255 ; au-dessus, deux octets par pixel
les pixels, ligne par ligne, de gauche à droite	chaque valeur en chiffres, suivie d'un espace ou d'un retour à la ligne : 0 255 0 255	chaque valeur sur un octet, sans séparateur : 00 ff 00 ff

L'en-tête est du texte dans les deux variantes. Même famille : PBM (p_1 , p_4), noir ou blanc, 0 ou 1 ; PPM (p_3 , p_6), trois valeurs par pixel. Documentation : netpbm.sourceforge.net/doc/pgm.html.

Image au format texte : PGM P2

images.ipynb · § 1

Le fichier `depart/motif.pgm` est au format P2. `read_text()` renvoie son contenu sous forme de texte. La bibliothèque Pillow ouvre le même fichier comme une image de 4×4 pixels.

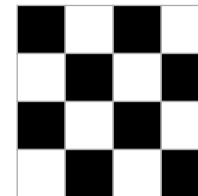
```
motif = Path("depart/motif.pgm")
print(motif.read_text())
image = Image.open(motif)
image.resize((160, 160), Image.NEAREST)
```

le chemin du fichier
affiche le texte : l'en-tête, puis les seize valeurs des pixels
ouvre l'image : size vaut (4, 4), mode vaut 'L' (niveaux de gris)
agrandit l'image 40 fois, sans lissage

`depart/motif.pgm`, 59 octets

L'image affichée par Pillow, agrandie

```
P2
4 4
255
0 255 0 255
255 0 255 0
0 255 0 255
255 0 255 0
```



La même image en binaire

§ 2

Pillow enregistre le .pgm en P5. Le fichier texte est lui aussi une suite d'octets : un par caractère, selon le code ASCII.

```
image.save(TRAVAIL / "motif.pgm")
octets = (TRAVAIL / "motif.pgm").read_bytes()
octets.hex(" ")
motif.read_bytes()[:23].hex(" ")
```

Pillow écrit la variante P5
lu comme des octets : `b'P5\n4 4\n255\n\x00\xff...'`
les mêmes octets, en hexadécimal
les 23 premiers octets du fichier texte

P2, 59 octets : l'en-tête et la première ligne de pixels

```
50 32 0a 34 20 34 0a 32 35 35 0a 30 20 32 35 35 20 30 20 32 35 35 0a
P 2  4  4  2  5  5  0  2  5  5  0  2  5  5  0
```

P5, 27 octets : l'en-tête et les seize pixels

```
50 35 0a 34 20 34 0a 32 35 35 0a 00 ff 00 ff ff 00 ff 00 00 ff 00 ff ff 00 ff 00
P 5  4  4  2  5  5  . . . . . . . . . . . . . . . .
```

En P2, 255 prend quatre octets : trois chiffres et un espace. En P5, un octet, ff.

À lire après la séance : § 3 à 6

Les sections 3 à 6 d'`images.ipynb` se lisent et s'exécutent après la séance. La séance reprend à la section 7.

Section	Contenu
§ 3 · Lire un fichier octet par octet	une fonction <code>hexdump</code> de six lignes : position, octets en hexadécimal, caractère
§ 4 · Trois formats réels pour la même image	les premiers octets d'un fichier (la signature) indiquent son format : P5, BM, \x89PNG
§ 5 · Une vraie image, en quatre formats	la taille d'une image non compressée se calcule (largeur × hauteur × octets par valeur) ; PNG et JPEG compressent
§ 6 · Le temps de lecture	lire les pixels depuis le fichier <code>P2</code> prend beaucoup plus longtemps que depuis le fichier <code>P5</code>

ASCII et UTF-8

§ 7

Un caractère est un nombre. ASCII en définit 128, sur un octet chacun : lettres sans accent, chiffres, ponctuation. UTF-8 garde ces 128 octets et écrit tous les autres caractères sur deux, trois ou quatre octets.

Quatre caractères

Caractère	ASCII	UTF-8
a	61	61
é	—	c3 a9
æ	—	c5 93
😊	—	f0 9f 98 80

Un mot, avec et sans ligature

Mot	len	UTF-8
œuf	3	c5 93 75 66, 4 octets
oeuf	4	6f 65 75 66, 4 octets

```
"œuf".encode("utf-8").hex(" ")  
"æ".encode("ascii")
```

les octets : len compte les caractères, encode les octets
UnicodeEncodeError : pas de code ASCII pour æ

æ manque aussi en ISO 8859-1, l'encodage des textes français avant UTF-8 : d'où « oeuf » dans les fichiers anciens.

Des noms de lieux

§ 7

Des noms de communes portent des caractères hors ASCII. Pour les écrire tels quels sur une carte, le fichier qui les porte est en UTF-8, et le programme qui le lit écrit `encoding="utf-8"`.

Commune	Lettre	Ce qu'un fichier ASCII peut écrire	Octets UTF-8
Œuilly (Aisne ; Marne)	œ	Oeuilly	7 pour 6 caractères
Plœuc-L'Hermitage (Côtes-d'Armor)	œ	Ploeuc-L'Hermitage	18 pour 17
L'Haÿ-les-Roses (Val-de-Marne)	ÿ	L'Hay-les-Roses	16 pour 15
Aÿ-Champagne (Marne)	ÿ	Ay-Champagne	13 pour 12

La dernière cellule du notebook compte, pour chaque nom, les caractères et les octets.

Du notebook au programme

Le code du notebook dans un fichier .py, lancé depuis un terminal ; ses trois paramètres passées via la ligne de commande

Le même code, dans un fichier

Copier dans un script python, fichier `recette.py`, le code des cellules, dans leur ordre (dernière version). `python recette.py` l'exécute en entier.

	Notebook	Script
Afficher	la dernière expression de la cellule	<code>print</code> ; une expression seule n'affiche rien
Dossier courant	celui du fichier <code>.ipynb</code>	celui du terminal
Les valeurs (recette, personnes, unités)	modifiées dans la cellule, puis la cellule relancée	passées sur la ligne de commande

Ce que l'interpréteur exécute

Python lit le fichier de haut en bas et exécute chaque instruction. Un `def` définit une fonction : son corps n'est pas exécuté à ce moment, mais à chaque appel.

```
import csv
NOM = "crepes"
def tableau(ingredients):
    lignes = ["| Ingrédient | Quantité |", "|---|---|"]
    ...
    return "\n".join(lignes)

table = tableau(ingredients)
```

exécuté : la bibliothèque est chargée
exécuté : la variable existe
exécuté : la fonction tableau existe
pas exécuté maintenant
l'appel : le corps de tableau s'exécute, puis la ligne suivante

Les fonctions sont écrites avant la première ligne qui les appelle : au moment de l'appel, elles existent.

Une fonction main

Les lignes du programme sont placées dans une fonction `main`. La fin du fichier appelle `main` seulement si le fichier est lancé avec `python recette.py`.

```
def lire_ingredients(chemin):           les fonctions utiles, sans changement
def main():                             le programme, dans une fonction
    ingredients = lire_ingredients(...)  les lignes du programme, indentées
if __name__ == "__main__":             vrai si le fichier est lancé avec python
    main()                             exécute le programme
```

Utilisation du fichier	<code>__name__</code> vaut	<code>main()</code> exécutée
<code>python recette.py</code> , dans le terminal	"__main__"	oui
<code>import recette</code> , dans un autre fichier Python	"recette"	non

Les arguments de la ligne de commande

argparse, bibliothèque standard, lit les arguments écrits après le nom du script : il les vérifie, les convertit, et construit l'aide de `--help`.

```
analyseur = argparse.ArgumentParser()
analyseur.add_argument("nom", choices=recettes_disponibles)
analyseur.add_argument("-p", "--personnes", type=int, default=4)
analyseur.add_argument("-u", "--unites", choices=("SI", "US"))
options = analyseur.parse_args()
options.nom, options.personnes, options.unites
```

Documentation : docs.python.org/fr/3/library/argparse.html.

l'analyseur des arguments
obligatoire ; une valeur de la liste
option ; convertie en entier ; 4 si absente
option ; deux valeurs possibles
lit la ligne de commande ; message et arrêt
si elle est invalide
les valeurs lues

<code>python recette.py pate_pizza -p 6 -u US</code>	nom, puis deux options
<code>python recette.py gaufres</code>	invalid choice, et la liste
<code>python recette.py --help</code>	l'aide, depuis les <code>add_argument</code>

Une ligne de commande pour la recette

Construire `recette.py` depuis les cellules du notebook : un fichier, une fonction `main`, `argparse`, un README ; un commit par étape

`cours3/3a_cli/` $\approx 45'$

Les étapes du TD

Le code du notebook devient un programme `recette.py`, lancé depuis un terminal. Chaque étape se termine par un commit.

Étape	Ce qu'on fait	Commits
0	préparer le dossier <code>travail/</code> et créer le dépôt git	0
1	écrire <code>recette.py</code> avec le code du notebook	1
2	mettre le programme dans une fonction <code>main</code>	2
3	les trois valeurs sur la ligne de commande, sur une branche	5
4	écrire un <code>README.md</code>	6

Le guide `guide_3a_cli.pdf`, dans le dossier `3a_cli/`, détaille chaque étape : le dossier où se placer, le code à écrire, les commandes et la façon de vérifier le résultat. Il contient aussi deux étapes à faire après la séance : ranger le code dans `src/` et les données dans `data/`, puis installer le programme comme une commande.

Une étape, un commit

Chaque étape se termine par un commit, avec les commandes du cours 2. L'étape 3 se fait sur une branche, fusionnée ensuite dans `master`.

<code>git status</code>	les fichiers modifiés depuis le dernier commit
<code>git diff</code>	les lignes modifiées dans <code>recette.py</code>
<code>git add recette.py</code>	ajoute le fichier au prochain commit
<code>git commit -m "Une fonction main"</code>	enregistre l'état ; le message nomme l'étape
<code>git checkout -b arguments</code>	crée la branche et s'y place
<code>git checkout master, git merge arguments</code>	revient sur master, y ramène les commits de la branche
<code>git log --oneline --graph</code>	un commit par ligne, les branches dessinées

`sortie/` est dans `.gitignore` : les fichiers produits par le programme ne sont pas versionnés, le programme les reproduit.

Étape 0 : le dossier de travail et le dépôt

`travail/` reçoit les données et devient un dépôt git. Le code n'y est pas encore : c'est l'étape 1.

	Ce qu'il faut faire	Ce que vous constatez
1	copier <code>depart/recettes/</code> et <code>depart/style.css</code> dans <code>travail/</code> ; dans VS Code, Fichier → Ouvrir le dossier <code>travail/</code> ; ouvrir un terminal Git Bash	
2	une fois par poste : <code>source /c/ProgramData/anaconda3/etc/profile.d/conda.sh</code> , puis <code>conda init bash</code> , et un nouveau terminal	
3	<code>git init</code> ; <code>echo "sortie/" > .gitignore</code>	

`sortie/` est le dossier que le programme va écrire : il ne sera pas versionné. `depart/` reste tel quel ; `depart/secours/recette.py` est le résultat attendu de l'étape 1, pour qui reste bloqué.

Étape 1 : le code du notebook dans un fichier

Un fichier `recette.py`, écrit dans l'éditeur, qui reprend les cellules du notebook `recette.ipynb` dans l'ordre, et produit `sortie/crepes.html`.

	Ce qu'il faut faire	Ce que vous constatez
1	en tête : les <code>import (csv, shutil, subprocess, Path)</code> ; <code>NOM = "crepes"</code> , <code>PERSONNES = 4</code> , <code>UNITES = "SI"</code> ; <code>RACINE = Path.cwd()</code> , puis <code>RECETTES</code> , <code>STYLE</code> et <code>SORTIE</code> déduits	
2	copier les fonctions utiles (section 1), le programme (section 3.3, avec <code>read_text</code> et <code>write_text</code>), l'appel de pandoc (section 4.4) ; finir par un <code>print</code> du chemin de la page	
3	<code>git add .</code> , puis <code>git commit -m "Le programme du notebook, dans un fichier"</code>	

Étape 2 : une fonction main

Les lignes du programme, sous les fonctions utiles, passent dans une fonction `main`, appelée en bas du fichier. Le fichier produit est le même qu'avant.

	Ce qu'il faut faire	Ce que vous constatez
1	écrire <code>def main()</code> : au-dessus des lignes du programme, et les indenter de quatre espaces	_____
2	en bas du fichier : <code>if __name__ == "__main__":</code> puis, indenté, <code>main()</code>	_____
3	<code>git diff</code> , puis <code>git commit -am "Une fonction main"</code>	_____

`git commit -am` ajoute les fichiers déjà suivis et valide en une commande. Un fichier nouveau demande `git add`.

Étape 3 : les arguments, sur une branche

Une branche `arguments`, et un commit par argument ajouté à `argparse` : la recette, puis le nombre de personnes, puis les unités. La branche est ensuite fusionnée dans `master`.

	Ce qu'il faut faire	Ce que vous constatez
1	<code>git checkout -b arguments ; dans main : l'analyseur, nom avec choices = les dossiers de recettes/ (boucle de la section 3.4), NOM = options.nom ; supprimer NOM en tête ; commit</code>	
2	<code>-p/--personnes, type=int, default=4 ; PERSONNES = options.personnes ; commit</code>	
3	<code>-u/--unites, choices=("SI", "US"), default="SI" ; UNITES = options.unites ; commit</code>	
4	<code>git checkout master, git merge arguments, git log --oneline --graph</code>	

Étape 4 : un README

Le dépôt reçoit un fichier `README.md` qui dit ce que fait le programme, comment l'installer et comment le lancer. Un modèle à compléter est dans `depart/modeles/`.

	Ce qu'il faut faire	Ce que vous constatez
1	copier <code>depart/modeles/README.md</code> dans <code>travail/</code> , l'ouvrir dans l'éditeur	
2	compléter : la phrase d'objectif, la façon de récupérer le dossier, un exemple avec <code>-p</code> et <code>-u</code> , la liste des recettes, votre nom	
3	<code>git add README.md, git commit -m "Un README"</code>	

Le README est le premier fichier lu par qui ouvre le dépôt : il dit comment installer et lancer. Le cours 6 le publie avec le dépôt.

À retenir

Un chemin relatif	part du dossier courant : celui du notebook, ou celui du terminal
<code>Path(__file__).parent</code>	le dossier du script, d'où qu'on le lance
<code>encoding="utf-8"</code>	dans chaque lecture et chaque écriture de texte
<code>subprocess.run(...)</code>	un programme externe, appelé depuis Python, en liste
<code>if __name__ == "__main__":</code>	<code>main()</code> s'exécute quand le fichier est lancé par <code>python</code> ; un <code>import</code> ne l'exécute pas
Un fichier binaire	des octets qui sont déjà les valeurs ; le texte les écrit en chiffres
Une signature	les premiers octets identifient le format ; l'extension le rappelle
Un caractère	un nombre ; ASCII en a 128, sur un octet ; UTF-8 écrit les autres sur deux à quatre
<code>argparse</code>	les valeurs sur la ligne de commande, vérifiées, et l'aide de <code>--help</code>
Un environnement conda	un Python et ses paquets ; <code>conda install</code> ajoute un paquet à l'environnement actif (TD 0a, selon les groupes)